Convolutional Neural Networks

Emre Ozfatura[1]

[1]Intelligent Systems Lab.
Sabanci University, Turkey

2 March 2026

Convolution

- The convolution, denoted $\circ$, between a signal $\ell_{\text{in}}[n]$ and the convolutional kernel $h[n]$ is defined as follows:

$$\ell_{\text{out}}[n] = h[n] \circ \ell_{\text{in}}[n] = \sum_{k=-\infty}^{\infty} h[n-k]\, \ell_{\text{in}}[k] \tag{1}$$

- The convolution computes the output values as a linear weighted sum. The weight between the input sample $\ell_{\text{in}}[k]$ and the output sample $\ell_{\text{out}}[n]$ is a function $h[n,k]$ that **depends only on their relative position** $n-k$. Therefore,

$$h[n,k] = h[n-k].$$

- If the signal $\ell_{\text{in}}[n]$ has a finite length N, then the sum is only done in the interval $k \in [0, N-1]$.

Convolution

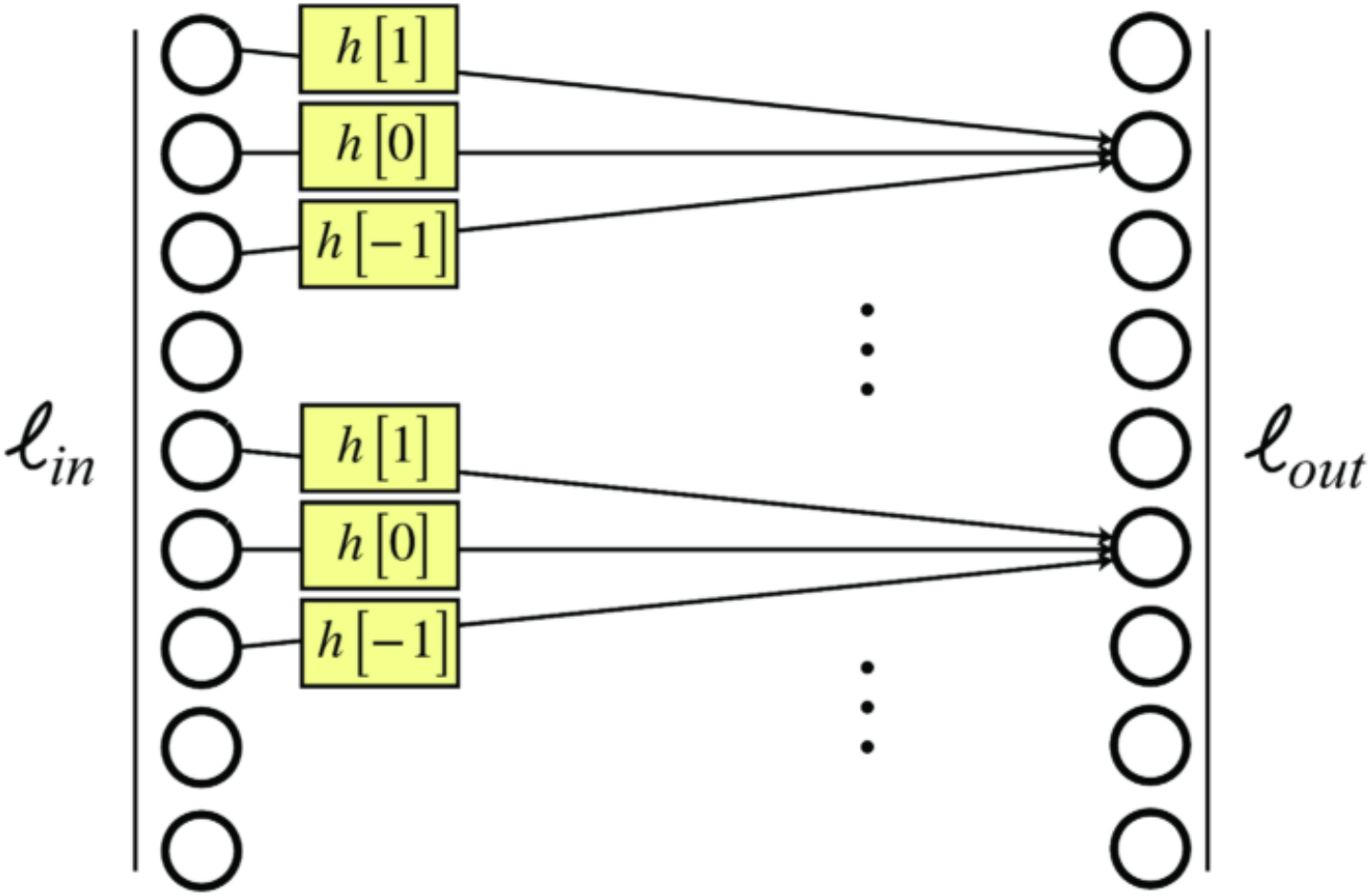


Figure 1: Illustration of 1d convolution and input-output relation, image from vision-book MIT
`https://visionbook.mit.edu/linear_image_filtering.html`

Convolution

One important property of the convolution is that it is commutative:

$$h[n] \circ \ell[n] = \ell[n] \circ h[n].$$

This property is easy to prove by changing the variables $k = n - k'$, so that:

$$h[n] \circ \ell_{\text{in}}[n] = \sum_{k=-\infty}^{\infty} h[n-k]\, \ell_{\text{in}}[k] \tag{2}$$

$$= \sum_{k'=-\infty}^{\infty} h[k']\, \ell_{\text{in}}[n-k'] \tag{3}$$

$$= \ell_{\text{in}}[n] \circ h[n]. \tag{4}$$

Convolution

- Convolution can be described as: first take the kernel $h[k]$ and mirror it to obtain $h[-k]$, then shift the mirrored kernel so that the origin is at location n.
- Next, multiply the input values around location n by the mirrored kernel and sum the result which is storred at $\ell_{\text{out}}[n]$.
- **Example:** Convolutional kernel has three non-zero values, $h[-1] = 1$, $h[0] = 2$, and $h[1] = 3$, then the output value at location n is

$$\ell_{\text{out}}[n] = 3\,\ell_{\text{in}}[n-1] + 2\,\ell_{\text{in}}[n] + 1\ell_{\text{in}}[n+1].$$

Convolution as matrix operation

- For finite-length signals, the convolution operation can be written in the form of matrix-vector multiplication.
- For instance, if the convolutional kernel h has only three non-zero values, then:

$$\begin{bmatrix} \ell_{\text{out}}[0] \\ \ell_{\text{out}}[1] \\ \ell_{\text{out}}[2] \\ \vdots \\ \ell_{\text{out}}[N-1] \end{bmatrix} = \begin{bmatrix} h[0] & h[-1] & 0 & \cdots & 0 \\ h[1] & h[0] & h[-1] & \cdots & 0 \\ 0 & h[1] & h[0] & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & h[0] \end{bmatrix} \begin{bmatrix} \ell_{\text{in}}[0] \\ \ell_{\text{in}}[1] \\ \ell_{\text{in}}[2] \\ \vdots \\ \ell_{\text{in}}[N-1] \end{bmatrix}.$$

- See PyTorch implementation `https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html`

Convolution: 1-dimensional to 2-dimensional

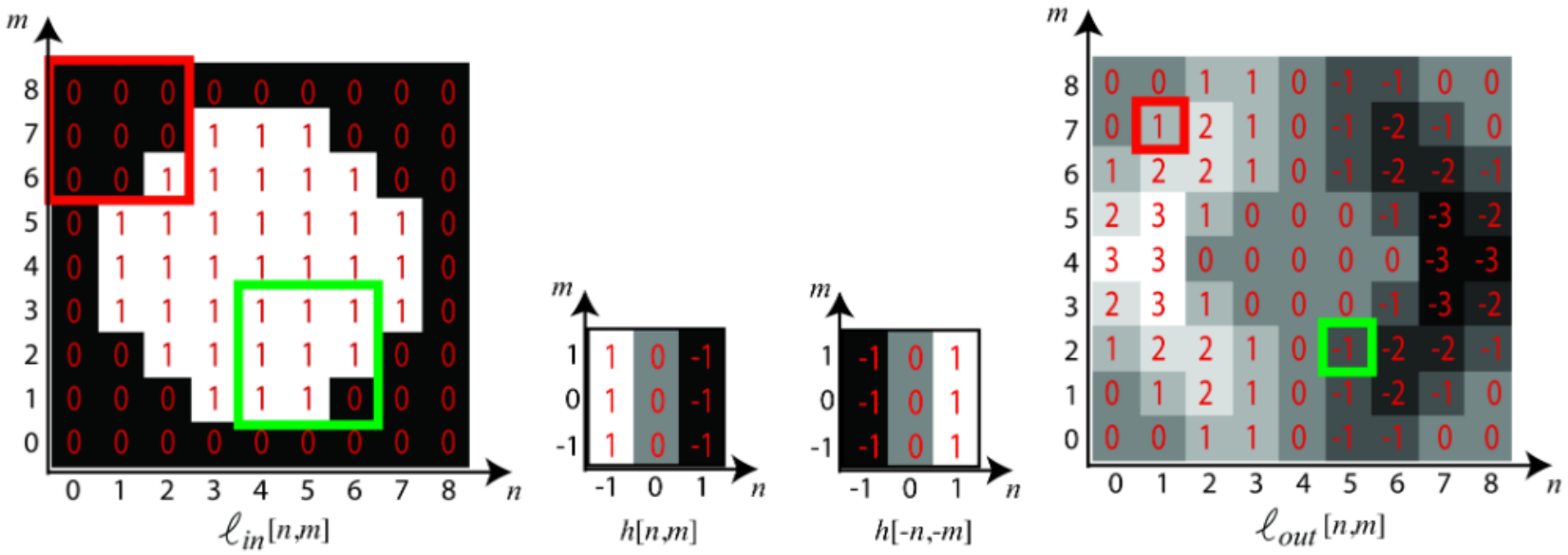


Figure 2: Illustration of simple 2D convolution from vision-book MIT, `https://visionbook.mit.edu/convolutional_neural_nets.html`

Note: Conv2d (Pytorch implementation of 2D Conv.) has slightly different mathematical expression (kernel is not centered and we use cross-correlation)

Convolution with filters

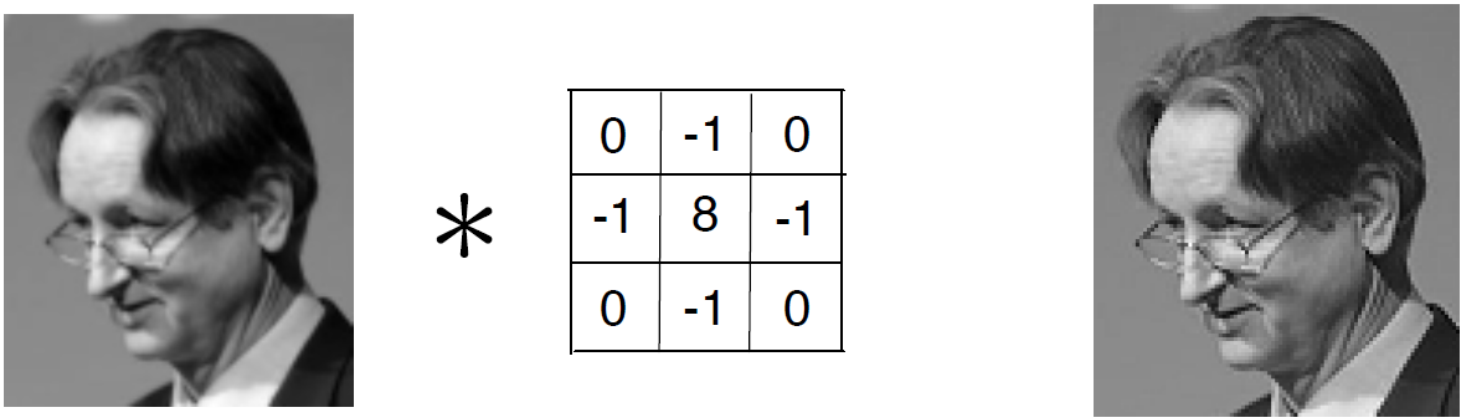


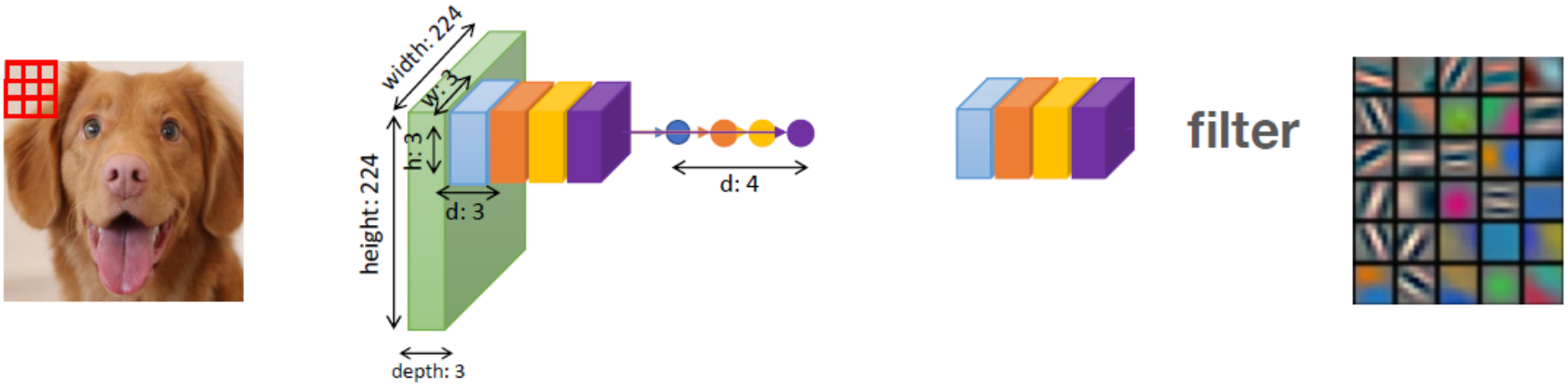
Figure 3: Illustration of image processing with filter (Image from CSC421)

Observe: Through filter we highlight the edges.

Why we use convolution?

- Similar to MLP, convolution also written as linear matrix-vector operation, then why use CNN?
- **Complexity:** Unlike MLP, in CNN, input-output relation is locally defined (sparse connection), hence, we have lower complexity (see example above).
- **Linear translation invariant (LTI):** "We don't know where within the image we expect to find any given item, so we often want to process the image in a spatially invariant manner, the same processing algorithm at every pixel", that is same kernel weights used over the image (shared weights).
- Further discussion: vision-book MIT, `https://visionbook.mit.edu/convolutional_neural_nets.html`

Image processing with convolutional neural networks



- **Objective:** Using convolutional filters extract features from the image over the NN layers.
- Local features (early layers) $\rightarrow$ global features (higher layers)

Conv2d: Implementing 2D convolution

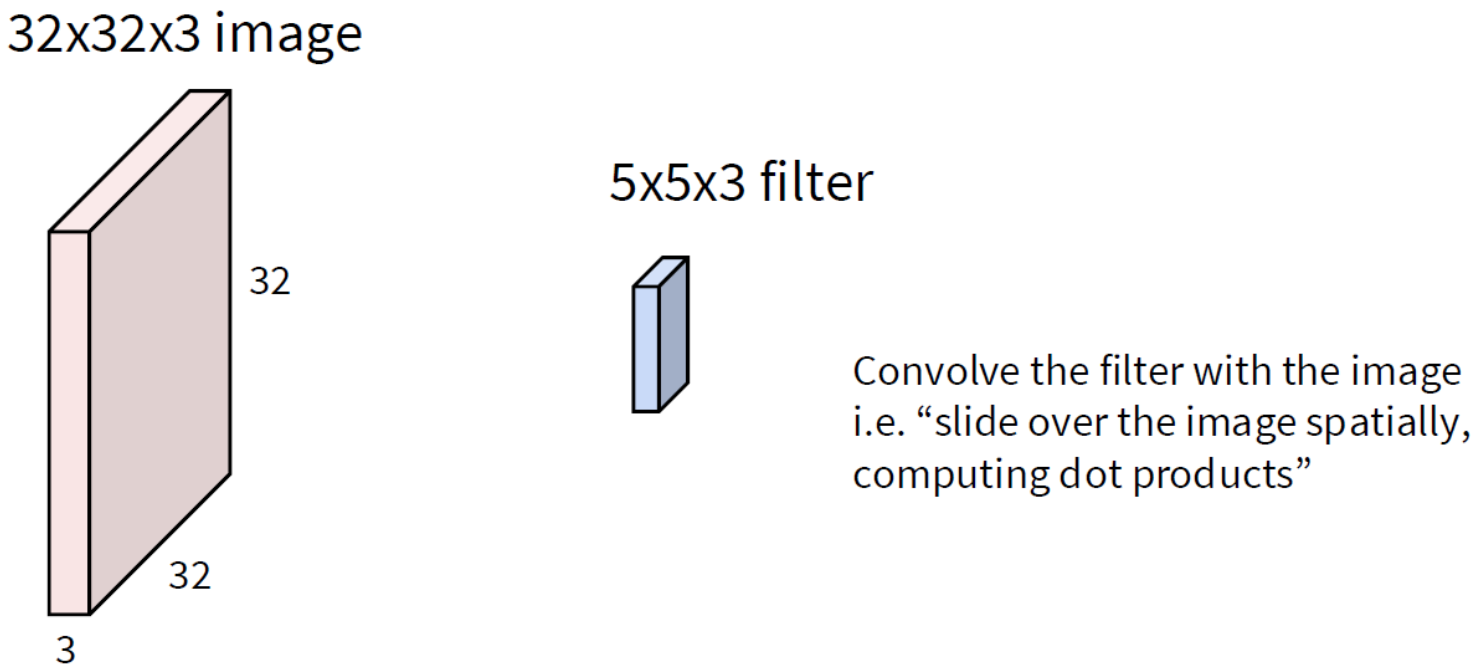


Figure 4: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: We have image of size $C_{\text{in}} = 3, H_{in} = 32, W_{in} = 32$, where C_{in} denotes the number of channels and $H_{in} =, W_{in}$ are two spatial dimensions height and width, respectively.

Conv2d: Implementing 2D convolution

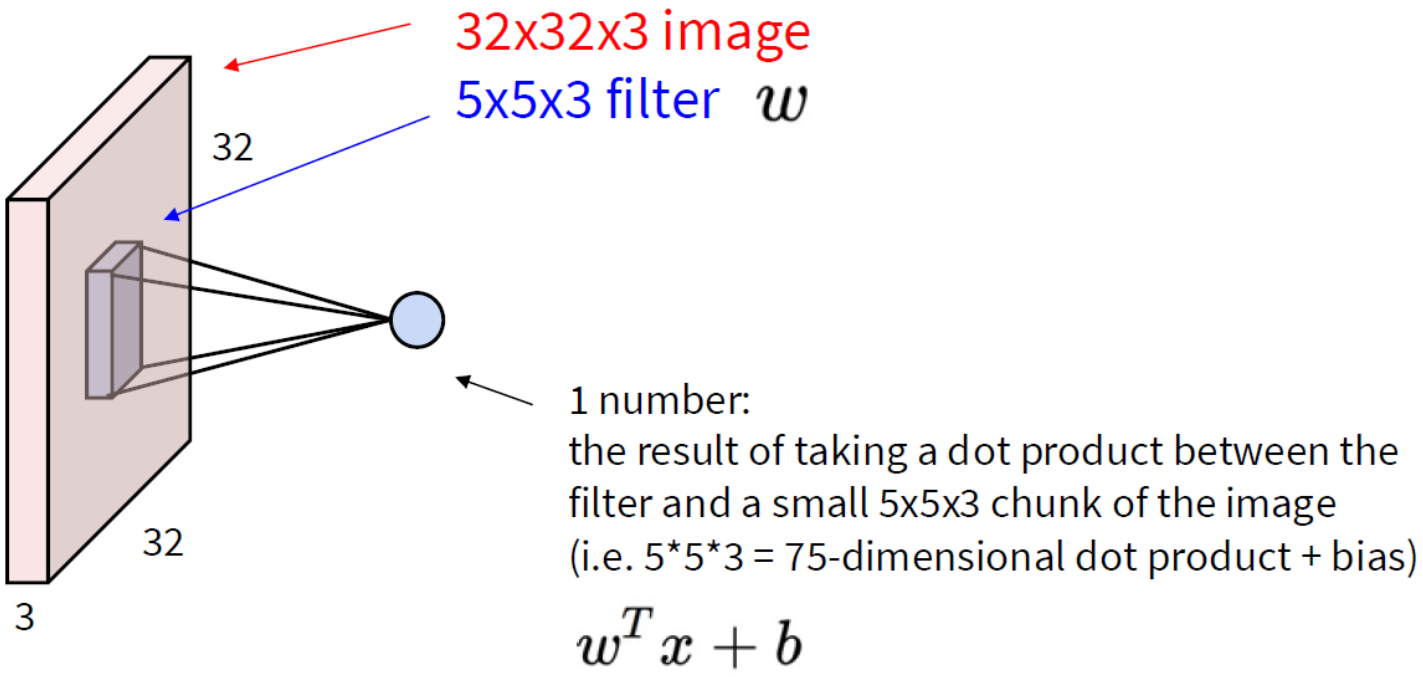


Figure 5: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: We have filter of size $C_{\text{in}} = 3, kh = 5, kw = 5$, where $[kh = 5, kw = 5]$ represents the kernel size.

Conv2d: Implementing 2D convolution

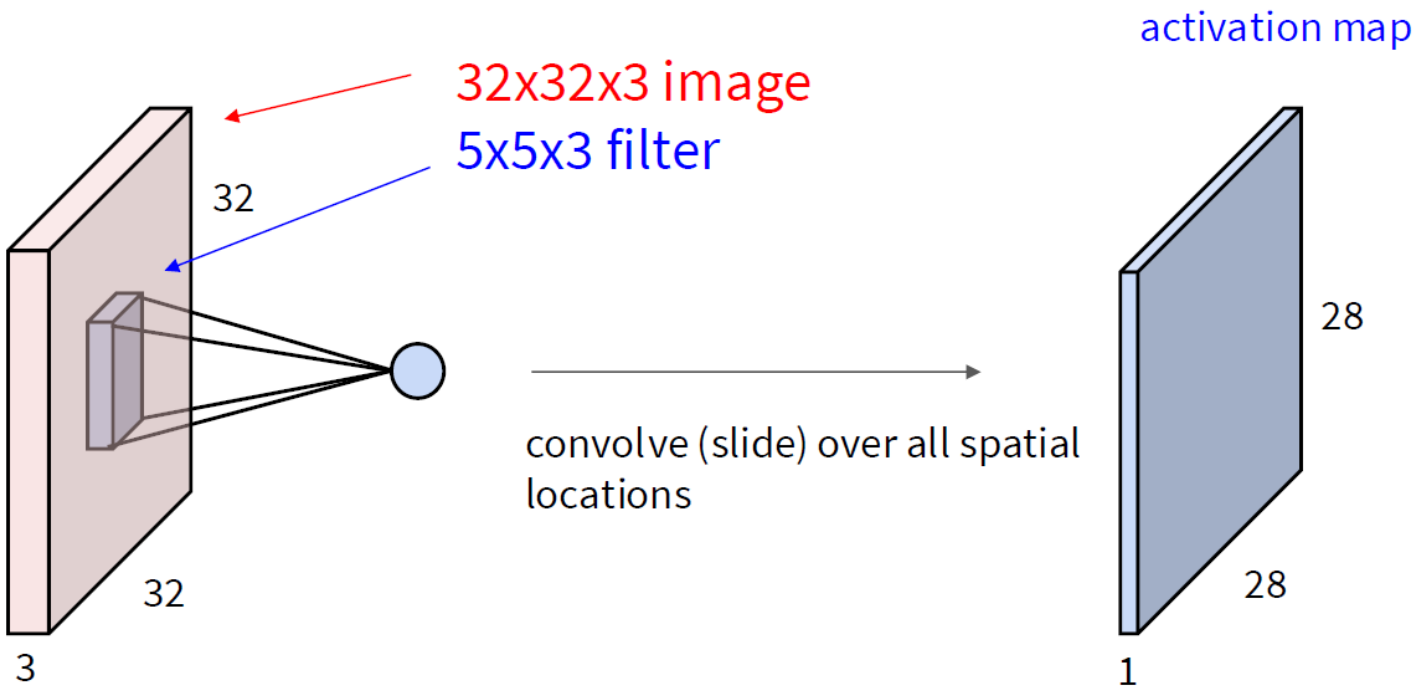


Figure 6: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: The number of channels in input and the filter are identical, and due to dot product output has single channel.

Conv2d: Implementing 2D convolution

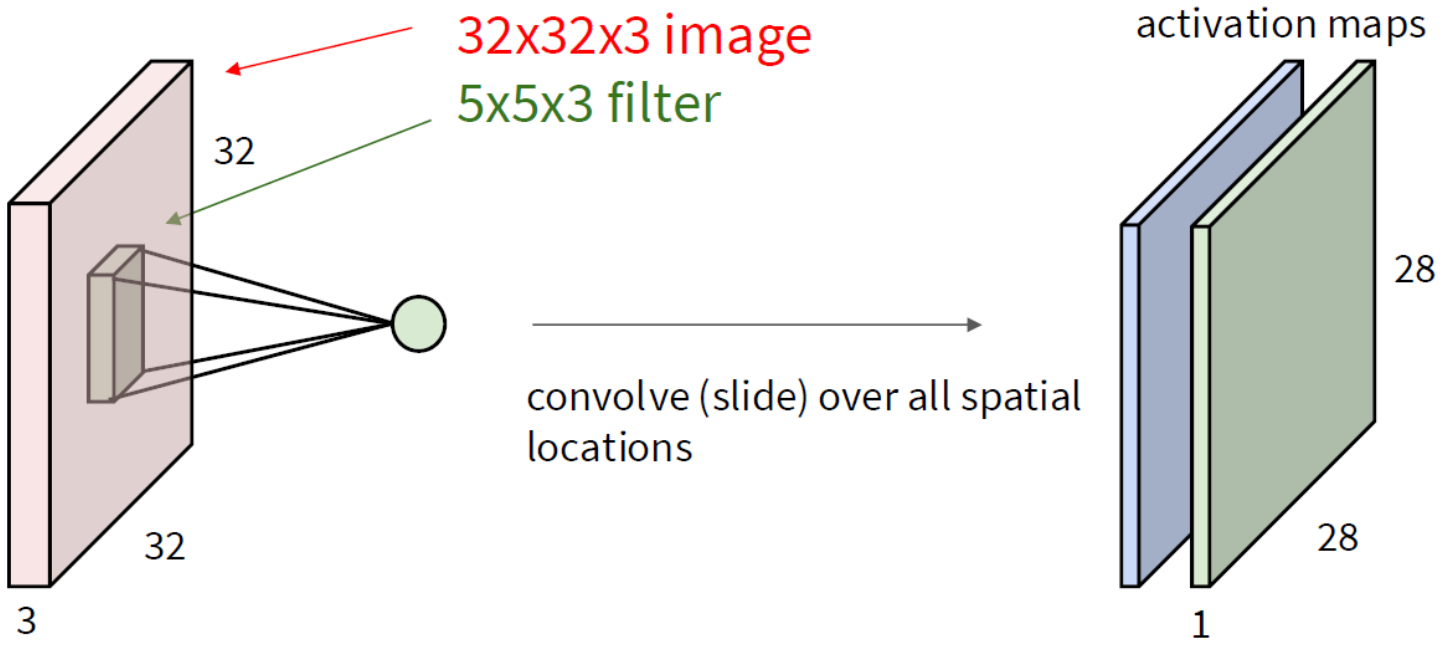


Figure 7: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: Now, if we use **2 filters** of size $C_{\text{in}} = 3, kh = 5, kw = 5$,the output will have two channels (denoted with blue and green), while spatial dimensions are same.

Conv2d: Implementing 2D convolution

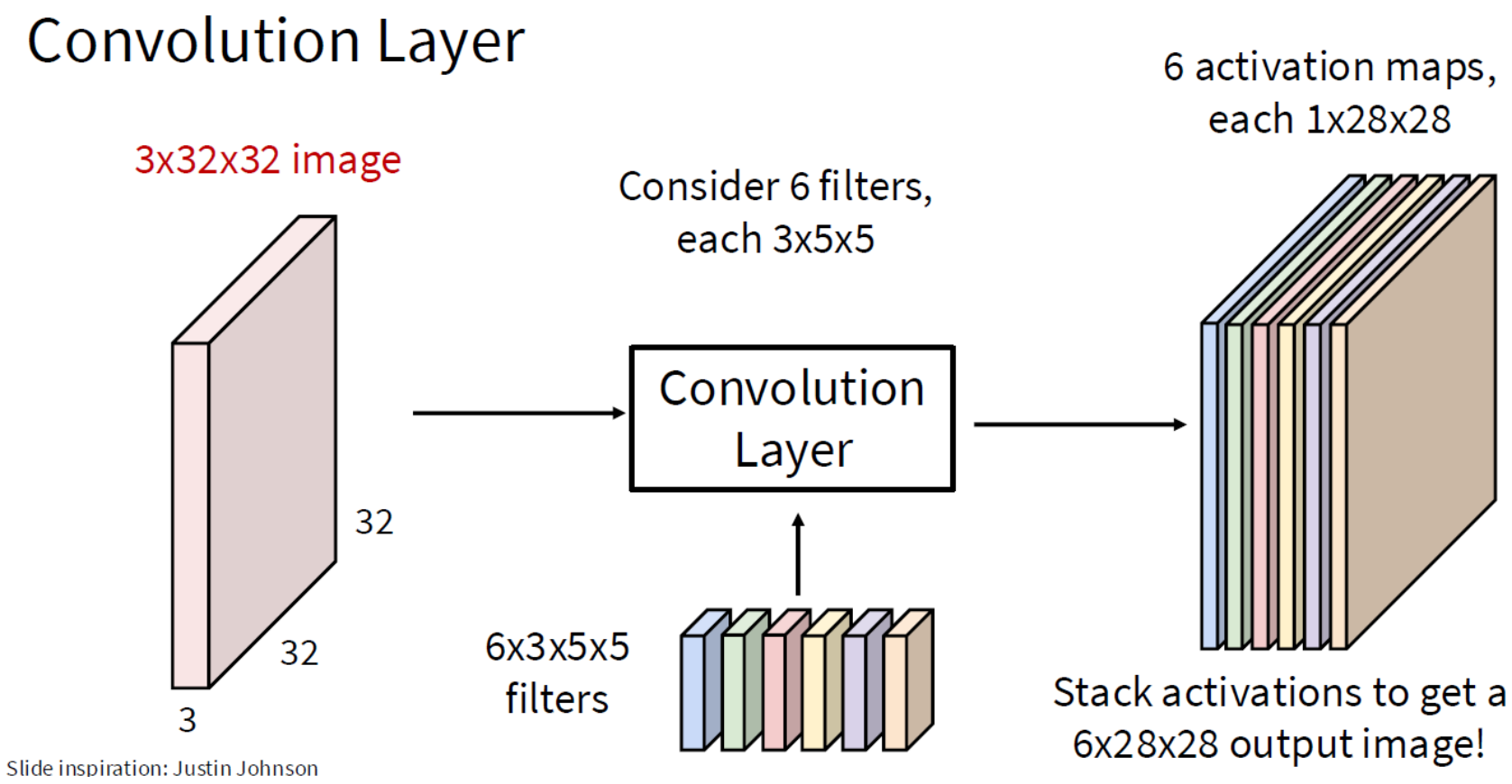


Figure 8: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: Number of output channels defined by the number of filters.

Conv2d: Implementing 2D convolution

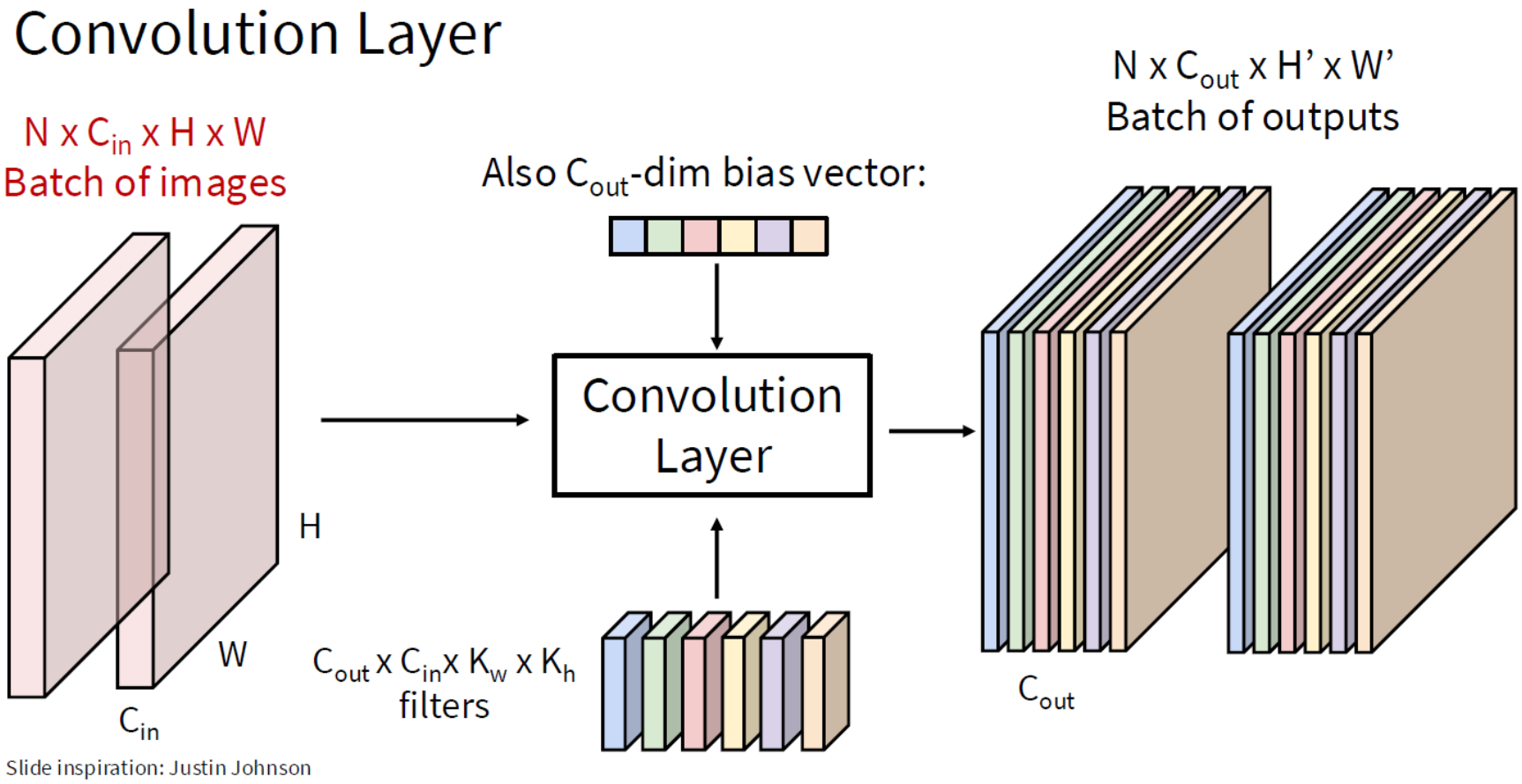


Figure 9: Illustration of input and the filter, image taken from Stanford CS231 course [L1].

Notice: If we have C_{out} number of output channel, it means we also need C_{out} dimensional bias vector.

Formal definition of Conv2D

- Applies a 2D convolution over an input signal composed of several input channels (depth).
- Input/output relation for given input $\mathbf{X}$ of size $(N, C_{\text{in}}, H_{in}, W_{in})$and output $\mathbf{Y}$ $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\mathbf{Y}[n, c_{in}, :] = \boldsymbol{B}[c_{out}, :] + \sum_{c_{in}=0}^{C_{\text{in}}-1} \mathbf{H}[c_{\text{out}}, c_{in}] \star \mathbf{X}[n, k, :] \tag{5}$$

$\star$ is 2D cross-correlation operator, N is a batch size, C_{in} denotes the number of input channels and H_{in}, W_{in} denotes the spatial dimensions, height and weight, respectively.

2D cross-correlation

- Let the input be $\mathbf{X} \in \mathbb{R}^{H_{in} \times W_{in}}$ and $\mathbf{H} \in \mathbb{R}^{kh \times kw}$ be the kernel.
- The 2D cross-correlation is defined as:

$$\mathbf{Y}[i, j] = \sum_{m=0}^{kh-1} \sum_{n=0}^{kw-1} \mathbf{H}[m, n]\, \mathbf{X}[i + m, j + n]. \tag{6}$$

- The 2D cross-correlation with dilation d:

$$\mathbf{Y}[i, j] = \sum_{m=0}^{kh-1} \sum_{n=0}^{kw-1} \mathbf{H}[m, n]\, \mathbf{X}[i + dm, j + dn]. \tag{7}$$

Dilated Convolution

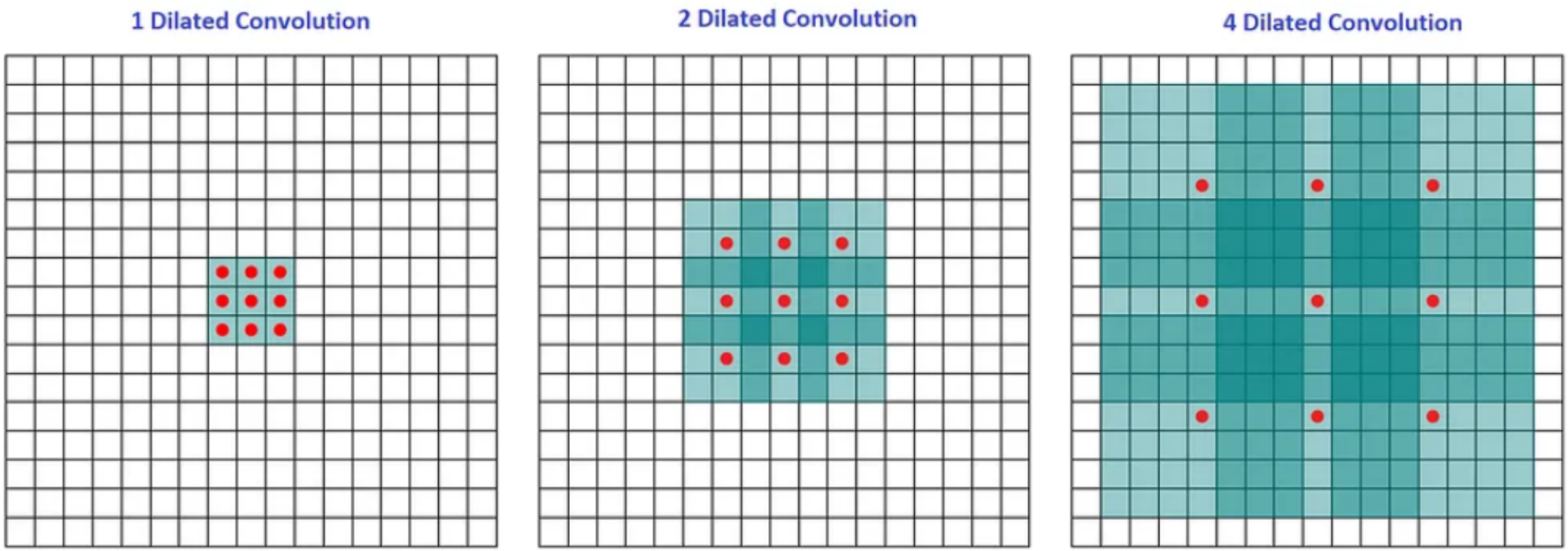


Figure 10: Illustration of dilated convolution.

With dilated convolution we increase the **receptive field**.

Conv2d:

- Let the input be $\mathbf{X} \in \mathbb{R}^{H \times W}$ and the kernel (filter) be $\mathbf{H} \in \mathbb{R}^{kh \times kw}$.
- The 2D cross-correlation is defined as:

$$\mathbf{Y}[i, j] = \sum_{m=0}^{kh-1} \sum_{n=0}^{kw-1} \mathbf{H}[m, n]\, \mathbf{X}[i + m, j + n]. \tag{8}$$

- The 2D cross-correlation with dilation d and stride s:

$$\mathbf{Y}[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{H}[m, n]\, \mathbf{X}[si + dm, sj + dn]. \tag{9}$$

Convolution with stride

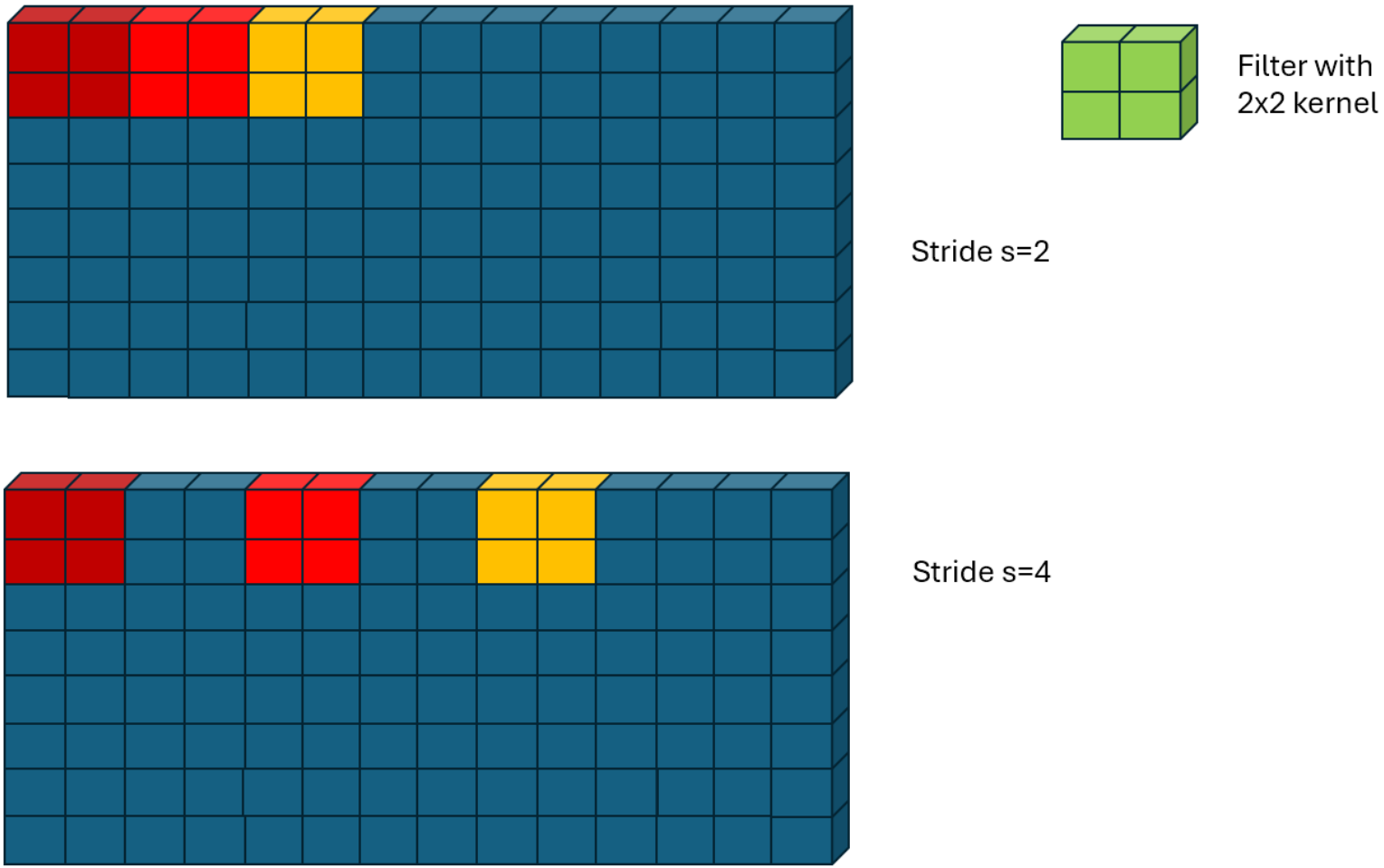


Figure 11: Illustration of convolution with stride $s = 2$ and $s = 4$.

Conv2d:

- The change of the spatial domain:

$$H_{\text{out}} = \left\lfloor \frac{H_{in} + 2p - d_h(k_h - 1) - 1}{s_h} + 1 \right\rfloor, \tag{10}$$

$$W_{\text{out}} = \left\lfloor \frac{W_{in} + 2p - d_w(h_w - 1) - 1}{s_w} + 1 \right\rfloor \tag{11}$$

- Filter size: $C_{in} \times C_{out} \times kh \times kw$.
- For PyTorch implementation, see `https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html`

Down-sampling on spatial domain (max-pooling)

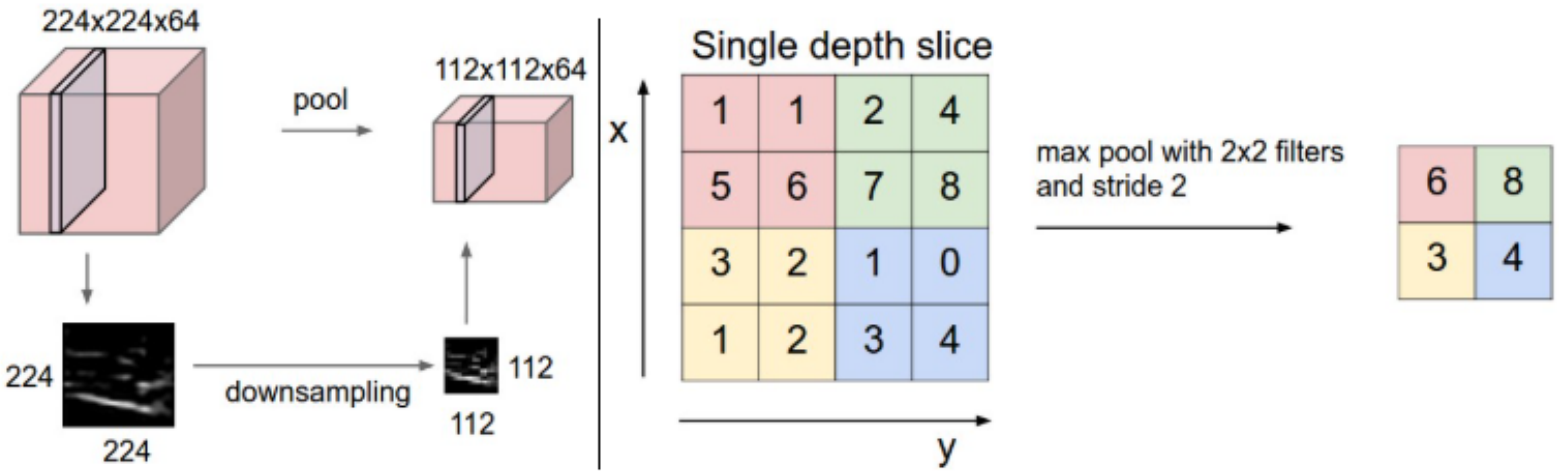


Figure 12: Illustration of max-pool down-sampling: Image from Stanford CS231 course
`https://cs231n.github.io/convolutional-networks/#pool`

Max-pool Vs. Average pooling

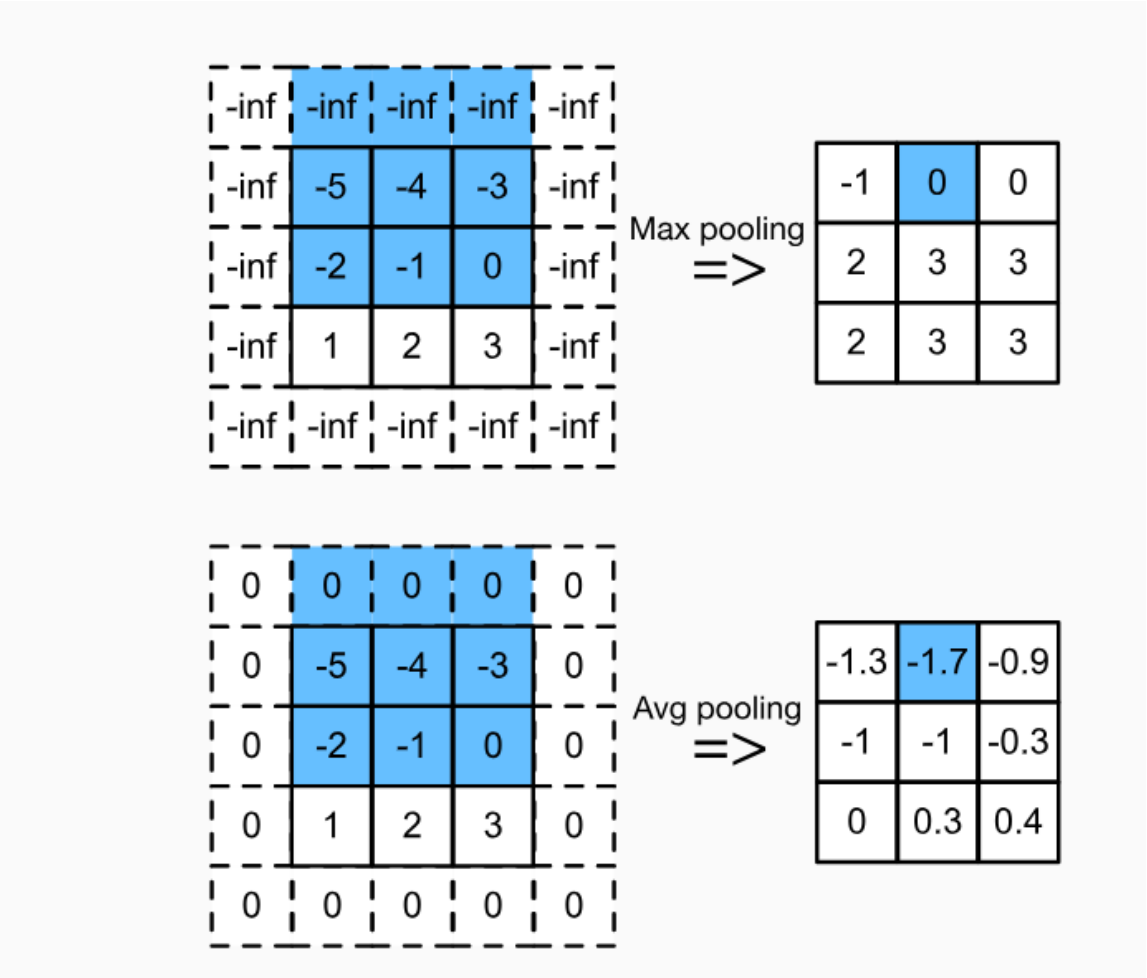


Figure 13: Comparison of pooling layers: Image from `https://tvm.d2l.ai/chapter_common_operators/pooling.html`

Deep vision architectures

- At each stage, different feature representations are learned through Conv2d operations.
- Over the layers, the spatial dimensions (height and width) are reduced by pooling operations or using Conv2d with stride/dilation.
- Often Conv2d with 3×3 kernel is employed for feature extraction, although larger kernels, like 7×7 can be used at initial layers.
- The learned features are stacked as channel variables (depth), through the process, the depth is increased in contrary to spatial dimensions.
- A fully connected layer (or cascaded multiple FC layers) is used following penultimate latent representation.

VGG Architecture

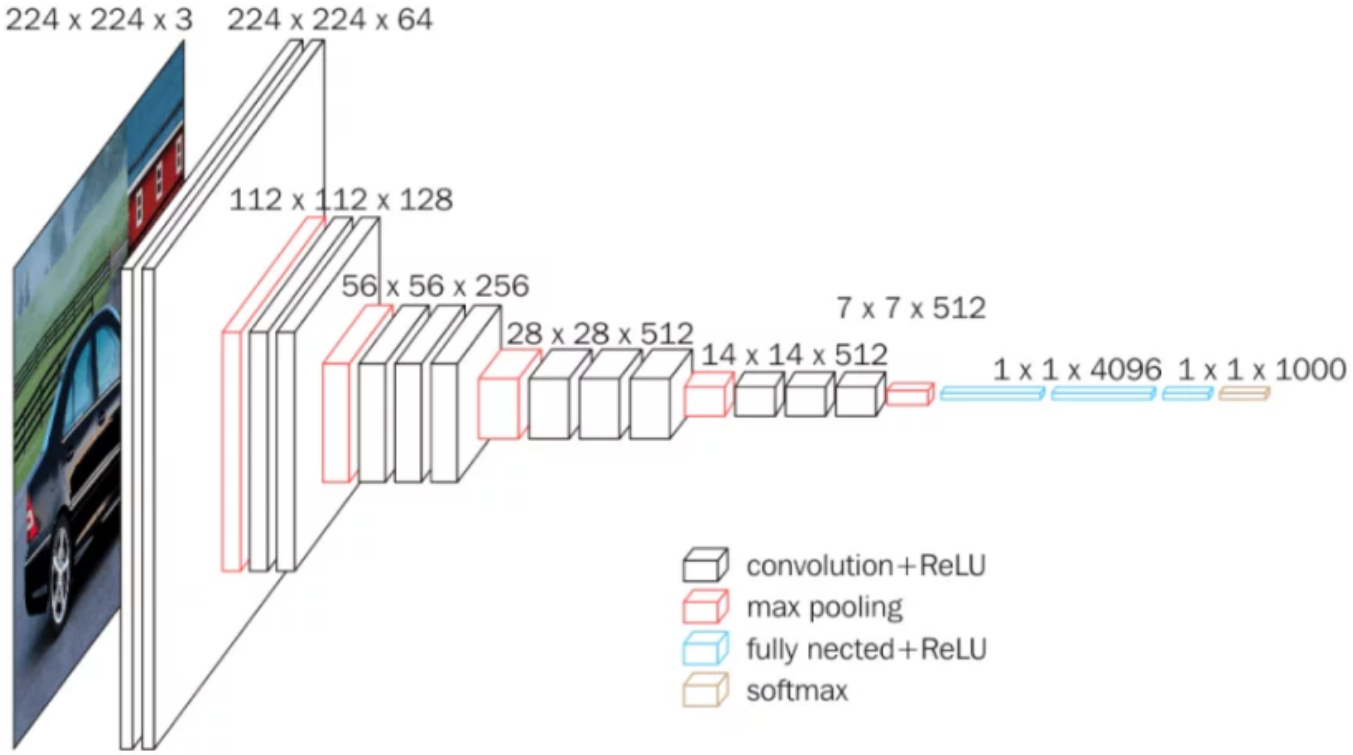


Figure 14: Illustration VGG-16 architecture, image from `https://neurohive.io/en/popular-networks/vgg16/`

Notice: Max-pooling is used to reduce the spatial dimension and through Conv2d the depth (the number of channels) is expanded.

VGG-16

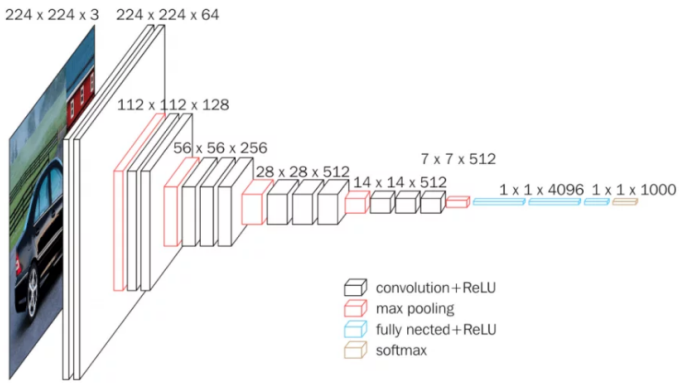


Figure 15: Illustration VGG-16 architecture, image from `https://neurohive.io/en/popular-networks/vgg16/`

- Consists of 13 convolutional and 3 fully connected layers.
- Convolution layers of 3x3 Conv with stride 1 and always used the same padding and maxpool layer of 2x2 filter of stride 2.
- **Remark:** Using 2x2 maxpool layer with stride 2 means reducing width and height to its half.

Resnet

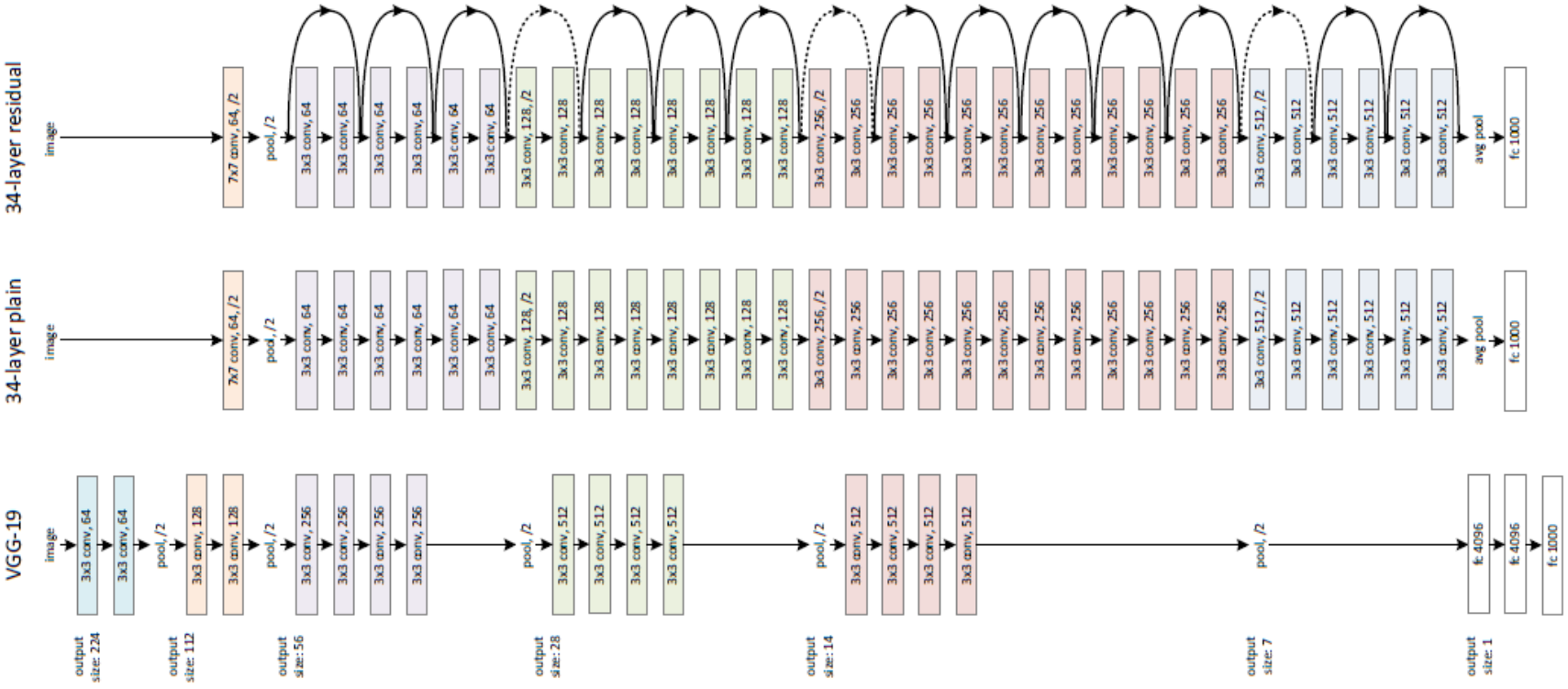


Figure 16: Resnet architecture

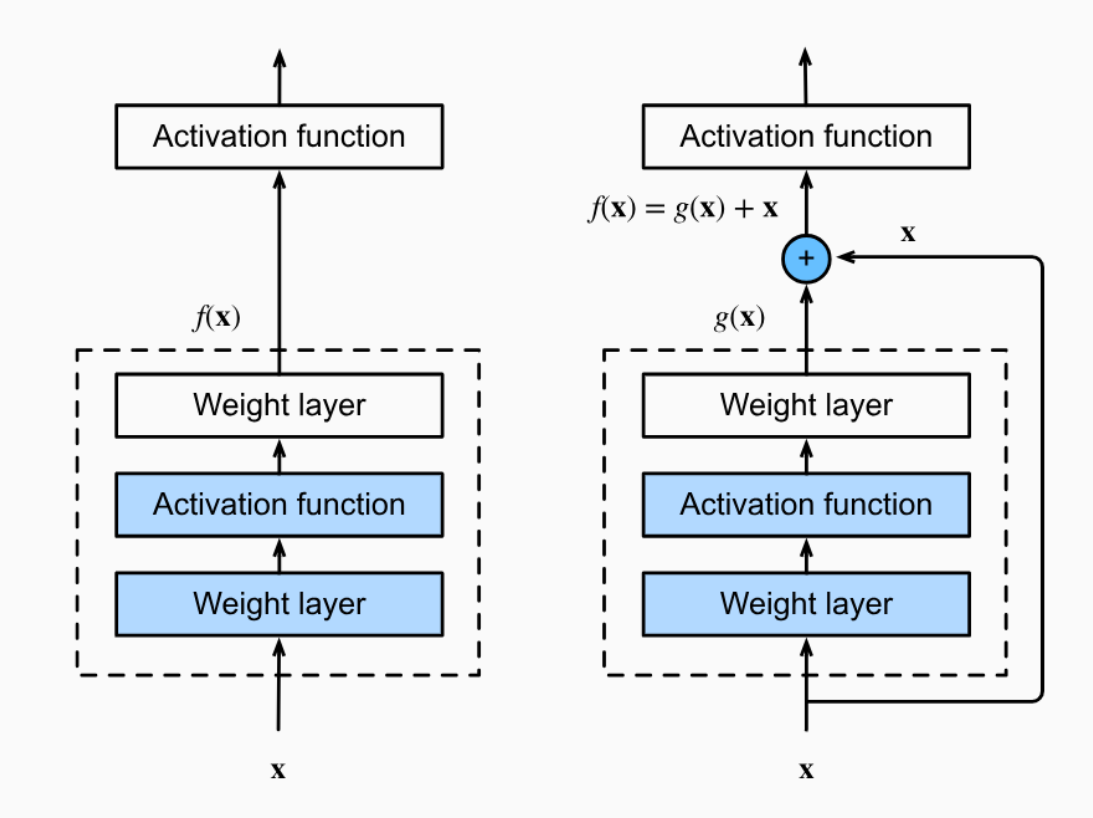


Figure 17: The residual concept: image from `https://d2l.ai/chapter_convolutional-modern/resnet.html`

Left, the dotted-line box must directly learn $f(\mathbf{x})$. Right, the portion within the dotted-line box needs to learn the **residual mapping**

$$g_\theta(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x},$$

ResNet: residual connection

- If the identity mapping $f(\mathbf{x}) = \mathbf{x}$ is the desired underlying mapping, the residual mapping amounts to

$$g_{\boldsymbol{\theta}}(\mathbf{x}) = 0$$

- Lets observe the gradient behavior:

$$\mathbf{y} = f(\mathbf{x}) = \mathbf{x} + g_{\boldsymbol{\theta}}(\mathbf{x})$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + \frac{\partial g_{\boldsymbol{\theta}}(\mathbf{x})}{\partial \mathbf{x}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\mathbf{I} + \frac{\partial g_{\boldsymbol{\theta}}(\mathbf{x})}{\partial \mathbf{x}} \right)$$

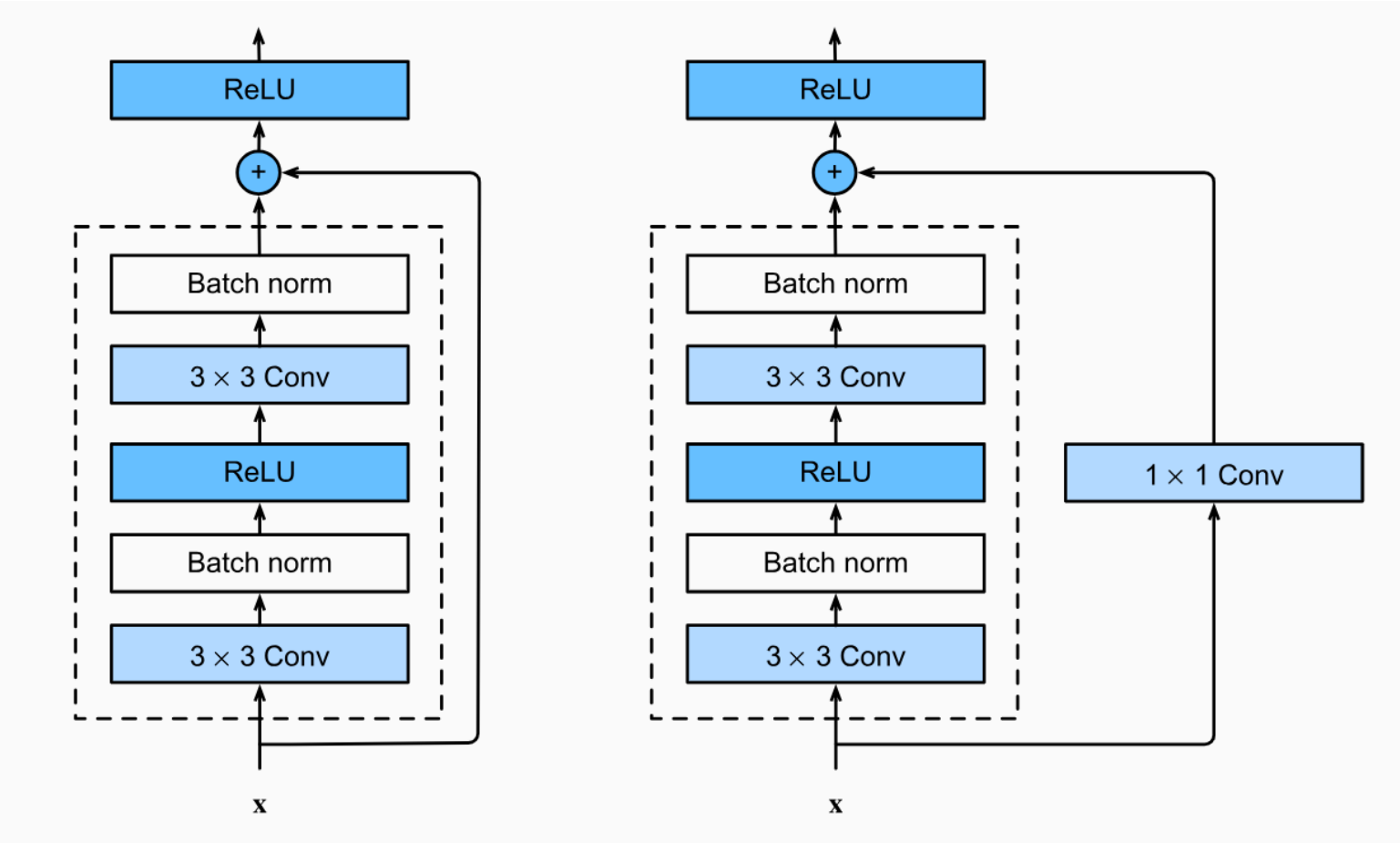


Figure 18: Illustration of residual block
https://d2l.ai/chapter_convolutional-modern/resnet.html

Residual block: 3x3 Conv → BN → ReLU (activation) → 3x3 Conv → BN. **Note:** for shortcut, dimensions (depth (numb of channels), height, width) should match!

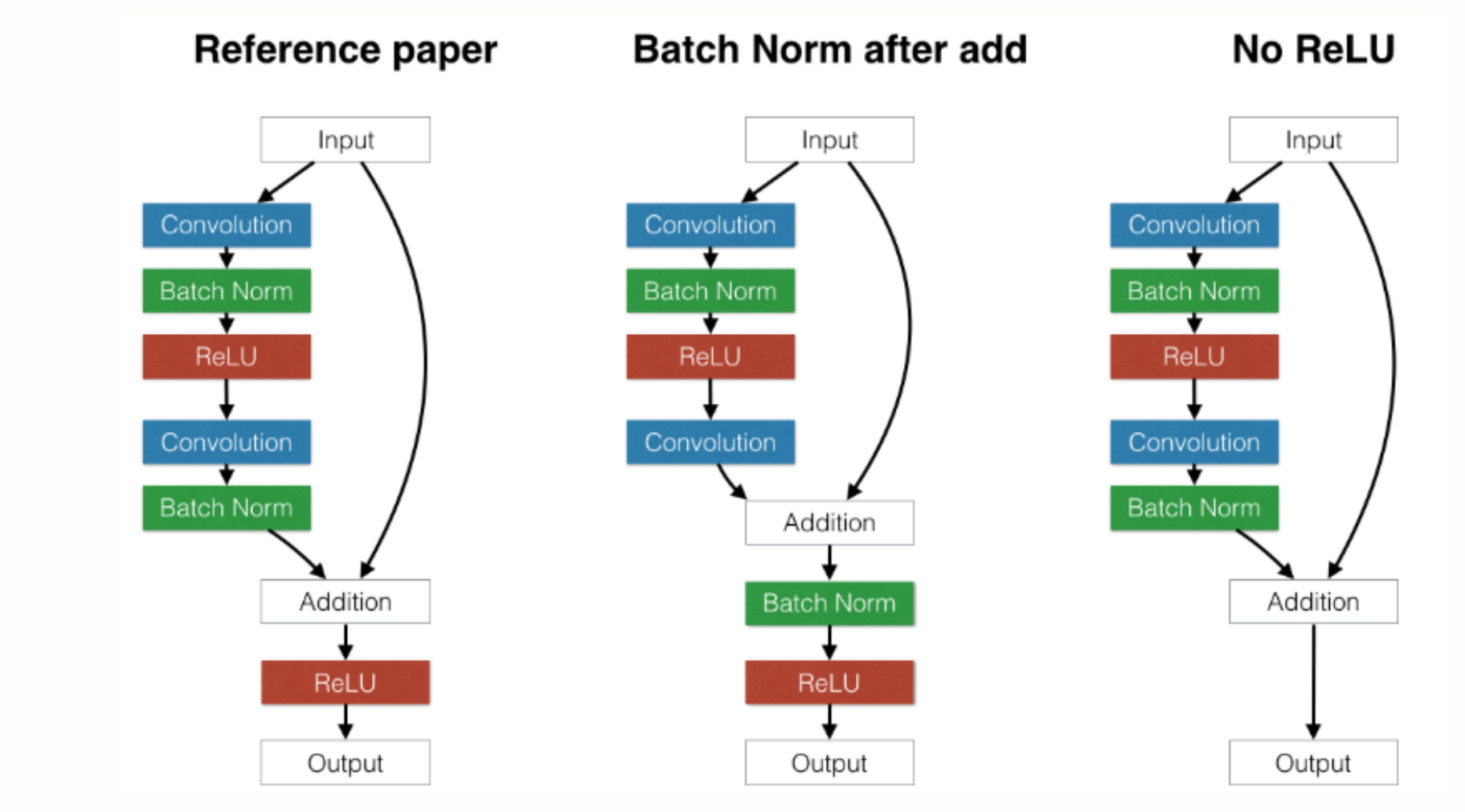


Figure 19: Different implementations of residual connection `http://torch.ch/blog/2016/02/04/resnets.html`.

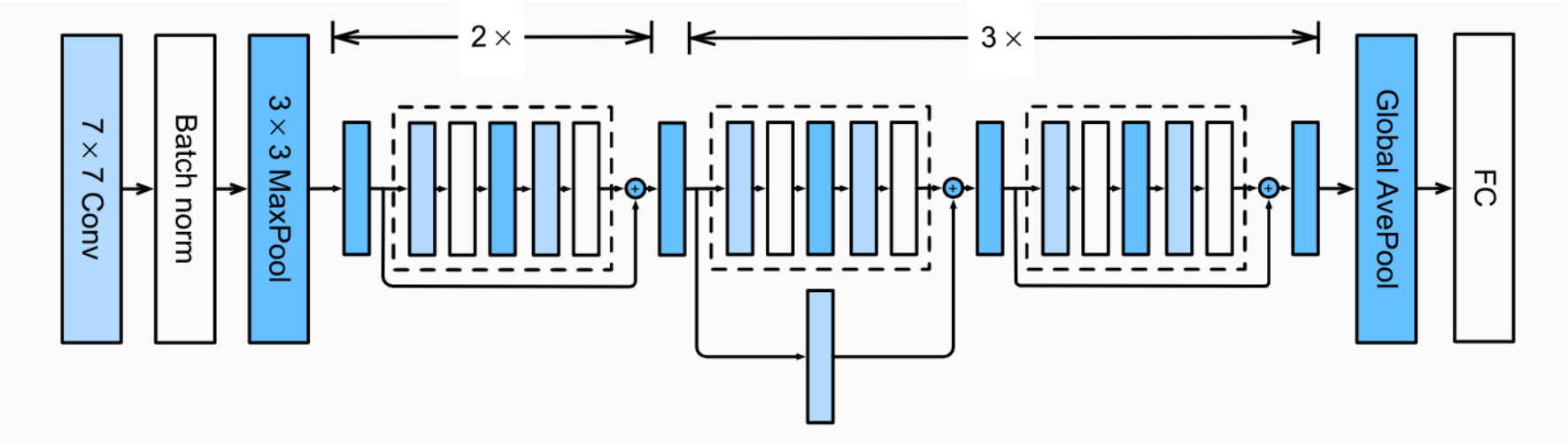


Figure 20: Illustration of ResNet-18 architecture.
`https://d2l.ai/chapter_convolutional-modern/resnet.html`

Notice: At initial stage we use larger kernel: 7x7 Conv, BN and maxpool, where the size of the spatial domain is reduced

Bottleneck block

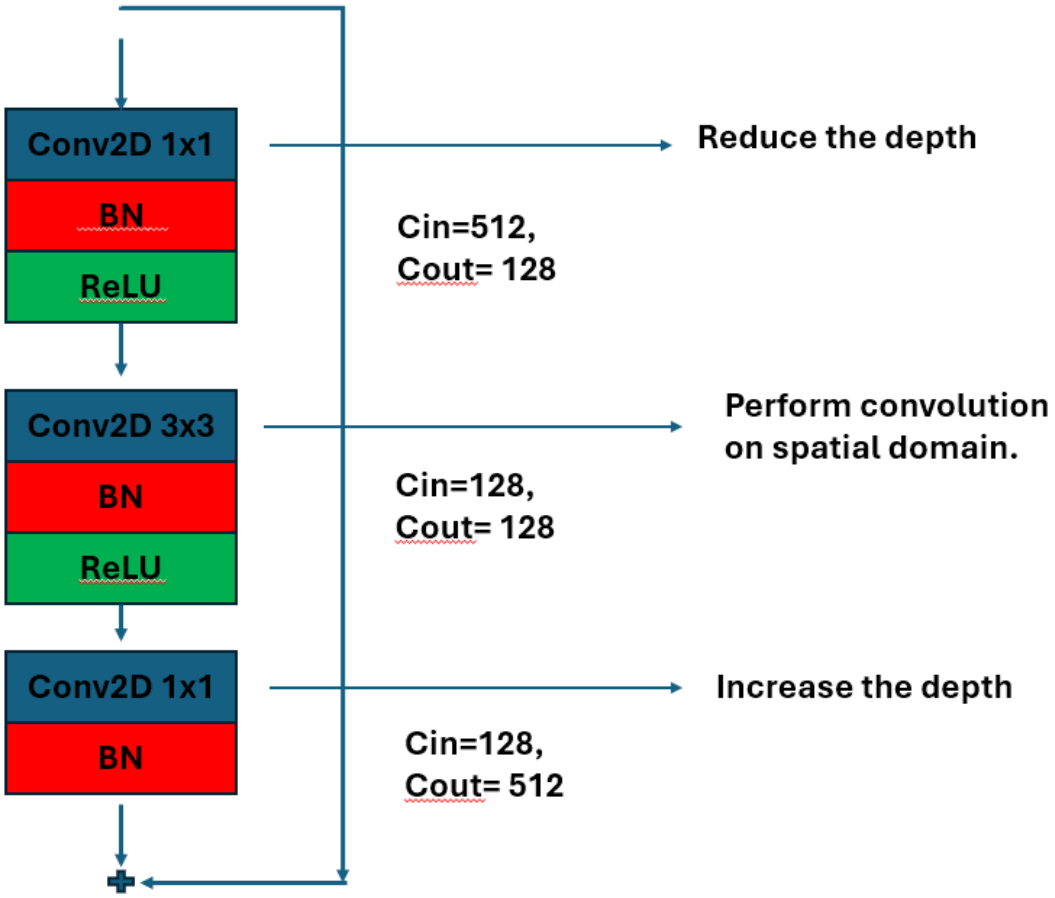


Figure 21: Bottleneck block illustration

Remark: when you see Conv2D/Conv with 1×1 kernel, it means we are performing up-sampling/down-sampling on the channel (depth) dimension.

Xavier initialization

- Let's look at the scale distribution of an output y_i for some fully connected layer with n_{in} inputs x_j and their associated weights w_{ij} for this layer i.e.,

$$y_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j. \tag{12}$$

- Assume the weights w_{ij} are all drawn independently from the same distribution with zero mean and variance σ^2.
- Also assume, the inputs to the layer x_j also have zero mean and variance γ^2 and they are independent of w_{ij} and independent of each other.

Xavier initialization

- In this case, we can compute the mean of y_i:

$$\mathbb{E}[y_i] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij} x_j] \tag{13}$$

$$= \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}]\mathbb{E}[x_j] \tag{14}$$

$$= 0. \tag{15}$$

- And the variance:

$$\mathrm{Var}[y_i] = \mathbb{E}[y_i^2] - (\mathbb{E}[y_i])^2 \tag{16}$$

$$= \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}^2 x_j^2] - 0 \tag{17}$$

$$= \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}^2]\mathbb{E}[x_j^2] \tag{18}$$

$$= n_{\text{in}} \sigma^2 \gamma^2. \tag{19}$$

Xavier initialization

- One way to keep the variance fixed is to set $n_{\text{in}}\sigma^2 = 1$. With the same reasoning, for the case of backpropagation, $n_{\text{out}}\sigma^2 = 1$ is also required to keep the variance fixed. Hence, the following equality

$$\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1. \tag{20}$$

  satisfy both constraints.
- Following the aforementioned logic, Xavier initialization samples weights from a Gaussian distribution with zero mean and variance

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Kaiming initialization

- Kaiming initialization also follows a similar logic and use zero mean Gaussian distribution with$\sigma^2 = \frac{2}{n_{\text{in}}}$

- **Question:** We know one of the drawbacks of ReLU is generating non-zero centered post activation values, then the assumption on x_j having zero mean is not true. How can we still prove $\sigma^2 = \frac{2}{n_{\text{in}}}$ works?

References: papers

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778)
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- Wu, Y., & He, K. (2018). Group normalization. In Proceedings of the European conference on computer vision (ECCV) (pp. 3-19).
- Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks In Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics.

References: online materials

- https://visionbook.mit.edu/convolutional_neural_nets.html
- https://github.com/FrancescoSaverioZuppichini/BottleNeck-InvertedResidual-FusedMBConv-in-PyTorch
- https://github.com/KaimingHe/deep-residual-networks
- https://d2l.ai/chapter_multilayer-perceptrons/numerical-stability-and-init.html

References: similar courses

- University of Toronto CSC 421 https://www.cs.toronto.edu/~rgrosse/courses/csc421_2019/slides/lec09.pdf
- Stanford CS231n: Deep Learning for Computer Vision https://cs231n.stanford.edu/slides/2025/lecture_5.pdf
- Berkeley CS182 https://cs182sp21.github.io/static/slides/lec-6.pdf